

Report Pratica

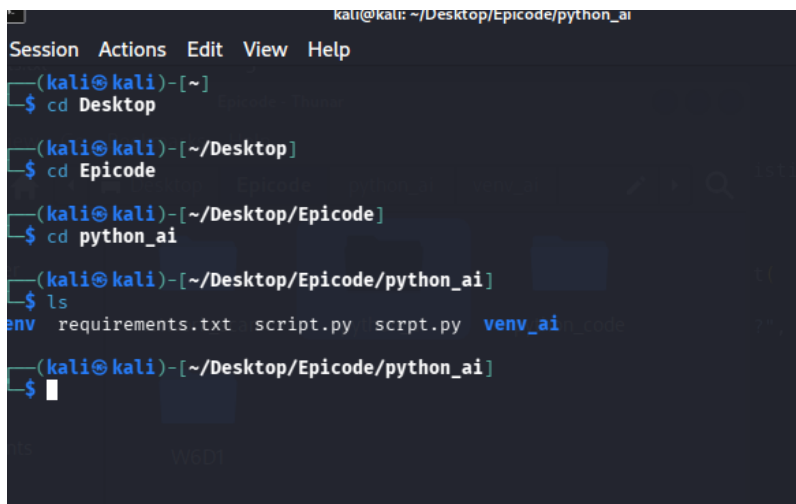
Realizzazione di una chatbot in Python con l'API Gemini di Google

Obiettivo della pratica

In questa pratica ho realizzato, passo dopo passo, un piccolo progetto Python (cartella python_ai) per interagire con l'intelligenza artificiale Gemini di Google tramite la libreria ufficiale google-genai. L'obiettivo era imparare a configurare un ambiente virtuale, gestire in modo sicuro una chiave API e scrivere uno script in grado di dialogare con il modello, per poi arrivare a una versione più avanzata con un ruolo (system instruction) personalizzato.

Preparazione dell'ambiente di lavoro

Per prima cosa mi sono posizionato nella cartella di progetto (Desktop/Epicode/python_ai) e ho verificato il contenuto della directory, dove erano già presenti l'ambiente virtuale venv_ai, gli script e il file dei requisiti:

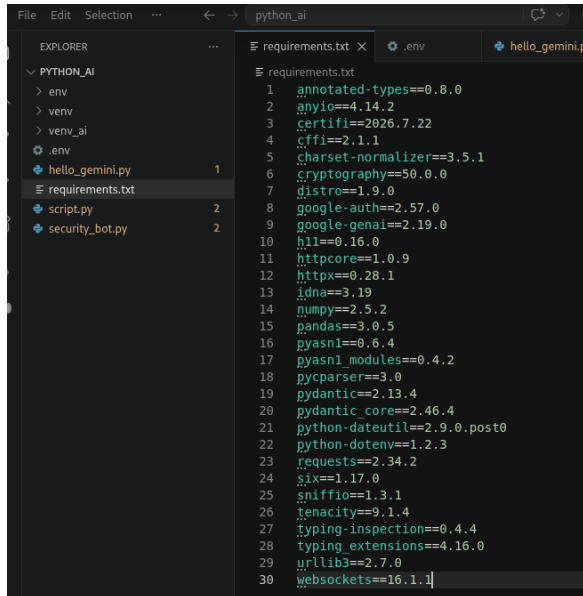


```
kali@kali: ~/Desktop/Epicode/python_ai
Session Actions Edit View Help
(kali@kali)-[~]
$ cd Desktop
(kali@kali)-[~/Desktop]
$ cd Epicode
(kali@kali)-[~/Desktop/Epicode]
$ cd python_ai
(kali@kali)-[~/Desktop/Epicode/python_ai]
$ ls
env  requirements.txt  script.py  script.py  venv_ai
```

Ho quindi attivato l'ambiente virtuale e generato il file requirements.txt con l'elenco dei pacchetti installati, così da poter riprodurre l'ambiente in futuro:

```
(venv_ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
• $ pip freeze >> requirements.txt

(venv_ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
• $ source venv_ai/bin/activate
```



```
File Edit Selection ... python_ai
EXPLORER
  PYTHON_AI
    > env
    > venv
    > venv_ai
    > .env
    hello_gemini.py
    requirements.txt
    script.py
    security_bot.py
requirements.txt
1 annotated-types==0.8.0
2 anyio==4.14.2
3 certifi==2026.7.22
4 cffi==2.1.1
5 charset-normalizer==3.5.1
6 cryptography==50.0.0
7 distro==1.9.0
8 google-auth==2.57.0
9 google-genai==2.19.0
10 h11==0.16.0
11 httpcore==1.0.9
12 httpx==0.28.1
13 idna==3.19
14 numpy==2.5.2
15 pandas==3.0.5
16 pyasn1==0.6.4
17 pyasn1_modules==0.4.2
18 pycparser==3.0
19 pydantic==2.13.4
20 pydantic_core==2.46.4
21 python-dateutil==2.9.0.post0
22 python-dotenv==1.2.3
23 requests==2.34.2
24 six==1.17.0
25 sniffio==1.3.1
26 tenacity==9.1.4
27 typing-inspection==0.4.4
28 typing_extensions==4.16.0
29 urllib3==2.7.0
30 websockets==16.1.1
```

Installazione delle librerie necessarie

Ho installato la libreria python-dotenv, utile per caricare variabili d'ambiente (come la chiave API) da un file .env senza scriverle direttamente nel codice:

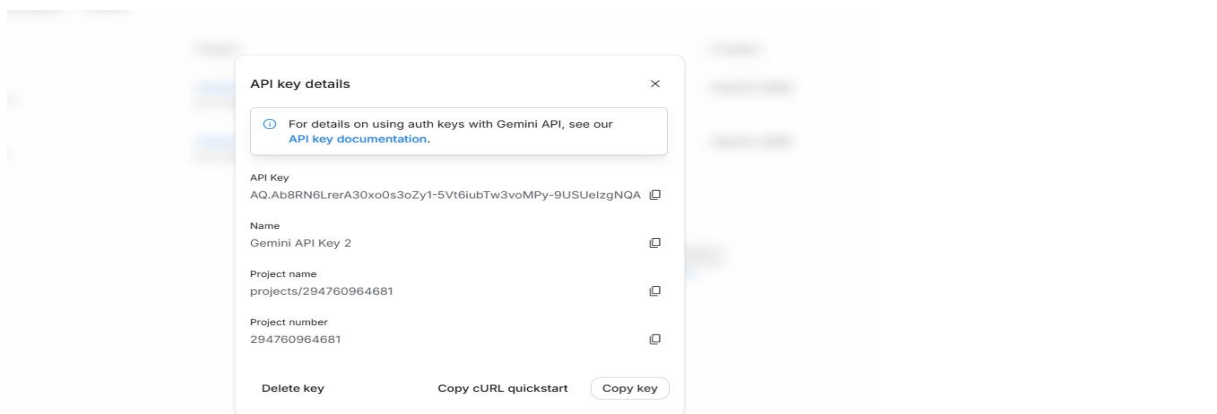
```
(venv_ai)-(kali@kali)-[~/Desktop/Epicode/python_ai]
• $ pip install python-dotenv
Collecting python-dotenv
  Downloading python_dotenv-1.2.3-py3-none-any.whl.metadata (29 kB)
  Downloading python_dotenv-1.2.3-py3-none-any.whl (22 kB)
Installing collected packages: python-dotenv
Successfully installed python-dotenv-1.2.3
```

Successivamente ho installato le librerie principali del progetto: google-genai (SDK ufficiale per Gemini), pandas e numpy:

```
Output Debug Console Terminal Ports
[venv ai]-(kali@kali):[~/Desktop/Epicode/python_ai]
$ pip install google-generativeai pandas numpy
Collecting google-generativeai
  Downloading google-generativeai-2.19.0-py3-none-any.whl.metadata (56 kB)
Collecting pandas
  Downloading pandas-3.0.5-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting numpy
  Downloading numpy-2.5.2-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (6.6 kB)
Collecting anyio<5.0.0,>=4.8.0 (from google-generativeai)
  Downloading anyio-4.14.2-py3-none-any.whl.metadata (4.6 kB)
Collecting google-auth<3.0.0,>=2.56.0 (from google-generativeai[requests]<3.0.0,>=2.56.0->google-generativeai)
  Downloading google-auth-2.57.0-py3-none-any.whl.metadata (6.0 kB)
Collecting httpx<1.0.0,>=0.28.1 (from google-generativeai)
  Downloading httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting pydantic<3.0.0,>=2.12.5 (from google-generativeai)
  Downloading pydantic-2.13.4-py3-none-any.whl.metadata (109 kB)
Collecting requests<3.0.0,>=2.28.1 (from google-generativeai)
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting tenacity<9.2.0,>=8.2.3 (from google-generativeai)
  Downloading tenacity-9.1.4-py3-none-any.whl.metadata (1.2 kB)
Collecting websockets<17.0,>=13.0.0 (from google-generativeai)
  Downloading websockets-16.1.1-cp313-cp313-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (6.8 kB)
Collecting typing-extensions<5.0.0,>=4.14.0 (from google-generativeai)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting distro<2,>=1.7.0 (from google-generativeai)
  Downloading distro-1.9.0-py3-none-any.whl.metadata (6.8 kB)
Collecting sniffio (from google-generativeai)
  Downloading sniffio-1.3.1-py3-none-any.whl.metadata (3.9 kB)
Collecting idna<=2.8 (from anyio<5.0.0,>=4.8.0->google-generativeai)
  Downloading idna-3.19-py3-none-any.whl.metadata (9.2 kB)
```

Creazione della chiave API su Google AI Studio

Per poter comunicare con il modello Gemini ho creato una chiave API dalla dashboard di Google AI Studio, nella sezione "API Keys":



La chiave generata è stata copiata e inserita in un file .env locale nella cartella del progetto (variabile GEMINI_API_KEY), in modo da poterla richiamare nello script tramite os.environ senza scriverla direttamente nel codice sorgente.

Nota di sicurezza: in alcuni degli screenshot originali la chiave API era visibile in chiaro. Per questo motivo non è stata riportata in questo documento; si raccomanda di rigenerarla dalla dashboard di Google AI Studio.

Primo script di prova (scrpt.py)

Come primo test ho scritto uno script minimale per verificare che la connessione con l'API funzionasse correttamente: caricamento delle variabili d'ambiente, creazione del client Gemini e invio di un semplice messaggio di saluto.

```
scrpt.py > ...
1  from google import genai
2  import os
3  from dotenv import load_dotenv
4
5  load_dotenv()
6
7  api_key = os.environ["GEMINI_API_KEY"]
8  client = genai.Client(api_key=api_key)
9
10 response = client.models.generate_content(
11     model="gemini-3.6-flash",
12     contents="ciao, come stai?",
13 )
14
15 print(response.text)
```

Eseguendo lo script dal terminale, il modello ha risposto correttamente al messaggio di test, confermando che la configurazione dell'ambiente e della chiave API era andata a buon fine:

```
(venv_3)~(kali@kali): ~/Desktop/Epicode/python_ai
$ python scrpt.py
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
Ciao! Sto bene, grazie! E tu come stai?
Come posso aiutarti oggi?
```

Script avanzato con ruolo personalizzato (script.py)

Una volta verificato il funzionamento di base, ho sviluppato una versione più completa dello script (script.py), in cui ho:

- Definito una system_instruction per assegnare al modello il ruolo di hacker etico e consulente di cybersecurity, focalizzato sull'individuazione di vulnerabilità e su consigli di difesa informatica
- Impostato la temperature a 0.2 per risposte più coerenti e meno "creative"
- Configurato i safety_settings per gestire le categorie di contenuto sensibile
- Utilizzato il modello gemini-3.6-flash per generare la risposta a un messaggio di test

```

1 from google import genai
2 import os
3 from dotenv import load_dotenv
4
5 load_dotenv()
6 client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
7
8
9 from google.genai import types
10 config = types.GenerateContentConfig(
11     system_instruction="""
12     sei un hacker etico e senior cybersecurity specialist Sei un Hacker Etico e consulente di Cybersecurity.
13     Individua vulnerabilità nel codice o nei sistemi e spiega come risolverle in modo semplice, rapido e sicuro.
14     Rimani sempre nei limiti dell'etica e della difesa informatica.
15     """,
16     temperature=0.2,
17 )
18
19 safety_settings = [
20     types.SafetySetting(category="HARM_CATEGORY_DANGEROUS_CONTENT", threshold="BLOCK_NONE"),
21     types.SafetySetting(category="HARM_CATEGORY_HATE_SPEECH", threshold="BLOCK_NONE"),
22     types.SafetySetting(category="HARM_CATEGORY_HARASSMENT", threshold="BLOCK_NONE"),
23     types.SafetySetting(category="HARM_CATEGORY_SEXUALLY_EXPLICIT", threshold="BLOCK_NONE"),
24 ]
25
26
27 response = client.models.generate_content(
28     model="gemini-3.6-flash",
29     contents="Com'è andata la tua vacanza?",
30     config=types.GenerateContentConfig(
31         safety_settings=safety_settings,
32         temperature=0.2
33     )
34 )
35
36 print(response.text)

```

Esecuzione dello script avanzato

All'esecuzione, lo script ha mostrato l'avviso informativo relativo all'uso diretto della funzione generate_content (suggerimento a usare Chat.send_message per usi futuri), seguito dalla risposta del modello, coerente con il ruolo impostato nella system instruction:

```

(kali@kali:~/Desktop/Epicode/python_ai)
$ python script.py
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
Essendo un'intelligenza artificiale, le mie "vacanze" consistono in qualche millisecondo di riposo tra una domanda e l'altra! 🤖

A parte gli scherzi, io sono sempre qui operativo e pronto ad aiutarti.

Piuttosto, raccontami della **tua**! Sei andato/a da qualche parte di bello o ti sei goduto/a un po' di relax a casa?
(kali@kali:~/Desktop/Epicode/python_ai)
$

```

Conclusioni

Attraverso questa pratica ho imparato a:

- Creare e gestire un ambiente virtuale Python con relativo file requirements.txt
- Proteggere le credenziali sensibili tramite file .env e la libreria python-dotenv
- Generare e utilizzare una chiave API di Google AI Studio per il modello Gemini
- Scrivere uno script base per interagire con un modello linguistico
- Personalizzare il comportamento del modello tramite system_instruction, temperature e safety_settings

Il progetto rappresenta una base solida per sviluppare, in futuro, un assistente virtuale più strutturato basato su Gemini.